Bài Nộp Cuối Kỳ - Khai Phá Dữ Liệu Credit Score

Notebook được xây dựng theo đúng trình tự mục bắt buộc của đề bài và được thiết kế để thực thi tuần tự từ trên xuống dưới. Để thực hiện đầy đủ các thí nghiệm phục vụ nộp bài, bật các cờ RUN_FULL_CLASSIFICATION và RUN_FULL_CLUSTERING.

1. Thư Viện và Cấu Hình

Thiết lập môi trường thực nghiệm, bao gồm thư viện, seed ngẫu nhiên và các đường dẫn dữ liệu chuẩn hóa.

In [1]:
```
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans, DBSCAN
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.svm import SVC

warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)

# Use False for fast skeleton run; set True for full assignment execution.
RUN_FULL_CLASSIFICATION = True
RUN_FULL_CLUSTERING = True

# Choose exactly one leakage strategy.
LEAKAGE_STRATEGY = "deduplicate_customer"  # options: deduplicate_customer, single_
SINGLE_MONTH_VALUE = None  # example: "January"

_cwd = Path.cwd()
PROJECT_ROOT = _cwd if (_cwd / 'data' / 'raw' / 'train.csv').exists() else None
if PROJECT_ROOT is None:
```

```
    for _p in _cwd.parents:
        if (_p / 'data' / 'raw' / 'train.csv').exists():
            PROJECT_ROOT = _p
            break
if PROJECT_ROOT is None:
    raise FileNotFoundError('Cannot locate project root containing data/raw/train.c

RAW_PATH = PROJECT_ROOT / 'data' / 'raw' / 'train.csv'
CLEAN_PATH = PROJECT_ROOT / 'data' / 'processed' / 'cleaned_credit_data.csv'
TARGET_COL = "Credit_Score"

ID_COLS = ["ID", "Customer_ID", "Month", "Name", "SSN"]
OPTIONAL_DROP_COLS = ["Type_of_Loan"]

print(f"SEED={SEED}")
print(f"RAW_PATH={RAW_PATH}")
print(f"CLEAN_PATH={CLEAN_PATH}")
print(f"LEAKAGE_STRATEGY={LEAKAGE_STRATEGY}")
print(f"PROJECT_ROOT={PROJECT_ROOT}")
```

```
SEED=42
RAW_PATH=d:\data-mining-credit-score\data\raw\train.csv
CLEAN_PATH=d:\data-mining-credit-score\data\processed\cleaned_credit_data.csv
LEAKAGE_STRATEGY=deduplicate_customer
PROJECT_ROOT=d:\data-mining-credit-score
```

2. Nạp Dữ Liệu Gốc

Nạp dữ liệu gốc và kiểm tra các đặc trưng cơ bản (kích thước, danh sách cột, nhãn mục tiêu).

In [2]:
```
if not RAW_PATH.exists():
    raise FileNotFoundError(f"Raw dataset not found: {RAW_PATH}")

df_raw = pd.read_csv(RAW_PATH)
print("Raw shape:", df_raw.shape)
print("Raw columns:", df_raw.columns.tolist())
if TARGET_COL in df_raw.columns:
    print("Target classes:", sorted(df_raw[TARGET_COL].dropna().astype(str).unique(
```

```
Raw shape: (100000, 28)
Raw columns: ['ID', 'Customer_ID', 'Month', 'Name', 'Age', 'SSN', 'Occupation', 'Ann
ual_Income', 'Monthly_Inhand_Salary', 'Num_Bank_Accounts', 'Num_Credit_Card', 'Inter
est_Rate', 'Num_of_Loan', 'Type_of_Loan', 'Delay_from_due_date', 'Num_of_Delayed_Pay
ment', 'Changed_Credit_Limit', 'Num_Credit_Inquiries', 'Credit_Mix', 'Outstanding_De
bt', 'Credit_Utilization_Ratio', 'Credit_History_Age', 'Payment_of_Min_Amount', 'Tot
al_EMI_per_month', 'Amount_invested_monthly', 'Payment_Behaviour', 'Monthly_Balanc
e', 'Credit_Score']
Target classes: ['Good', 'Poor', 'Standard']
```

Giả Định và Ràng Buộc

- Chiến lược chống rò rỉ: `deduplicate_customer` (mỗi `Customer_ID` chỉ giữ một hồ sơ).

- Cột định danh bắt buộc loại bỏ: `ID`, `Customer_ID`, `Month`, `Name`, `SSN`.
- `Type_of_Loan` được loại bỏ theo khuyến nghị do là free-text khó chuẩn hóa.
- PCA chỉ dùng continuous features và chuẩn hóa bằng `StandardScaler` trước khi fit.
- Classification dùng 3 mô hình hợp lệ, `GridSearchCV`, và `StratifiedKFold(n_splits=10)`.
- Độ đo so sánh classification: `F1-macro`.
- Clustering không dùng nhãn làm input, đánh giá bằng ARI/NMI với nhãn gốc.
- Mục tiêu tái lập: notebook chạy tuần tự từ trên xuống dưới với seed cố định.

3. Tiền Xử Lý và Chống Rò Rỉ Dữ Liệu

Thực hiện tiền xử lý dữ liệu theo một chiến lược chống rò rỉ dữ liệu duy nhất, đồng thời chuẩn hóa và mã hóa thuộc tính phục vụ mô hình hóa.

In [3]:
```
df_prep = df_raw.copy()
shape_before = df_prep.shape

# Leakage handling: choose exactly one strategy.
if LEAKAGE_STRATEGY == "single_month":
    if SINGLE_MONTH_VALUE is None:
        raise ValueError("Set SINGLE_MONTH_VALUE when LEAKAGE_STRATEGY='single_mont
    if "Month" not in df_prep.columns:
        raise ValueError("Month column not found for single_month strategy.")
    df_prep = df_prep[df_prep["Month"].astype(str) == str(SINGLE_MONTH_VALUE)].copy
elif LEAKAGE_STRATEGY == "deduplicate_customer":
    if "Customer_ID" not in df_prep.columns:
        raise ValueError("Customer_ID column not found for deduplicate strategy.")
    df_prep = df_prep.drop_duplicates(subset=["Customer_ID"], keep="first").copy()
else:
    raise ValueError("Unsupported LEAKAGE_STRATEGY.")

shape_after_leakage = df_prep.shape

drop_cols = [c for c in (ID_COLS + OPTIONAL_DROP_COLS) if c in df_prep.columns]
df_prep = df_prep.drop(columns=drop_cols, errors="ignore")

continuous_candidates = [
    "Age",
    "Annual_Income",
    "Monthly_Inhand_Salary",
    "Num_Bank_Accounts",
    "Num_Credit_Card",
    "Interest_Rate",
    "Num_of_Loan",
    "Delay_from_due_date",
    "Num_of_Delayed_Payment",
    "Changed_Credit_Limit",
    "Num_Credit_Inquiries",
    "Outstanding_Debt",
    "Credit_Utilization_Ratio",
    "Total_EMI_per_month",
```

```
    "Amount_invested_monthly",
    "Monthly_Balance"
]

for col in continuous_candidates:
    if col in df_prep.columns:
        df_prep[col] = pd.to_numeric(
            df_prep[col]
            .astype(str)
            .str.replace("_", "", regex=False)
            .str.replace(",", "", regex=False),
            errors="coerce"
        )

if "Age" in df_prep.columns:
    df_prep = df_prep[df_prep["Age"].fillna(0) > 0].copy()

non_negative_cols = [
    "Num_Bank_Accounts",
    "Num_Credit_Card",
    "Num_of_Loan",
    "Num_Credit_Inquiries",
    "Delay_from_due_date",
    "Num_of_Delayed_Payment"
]
for col in non_negative_cols:
    if col in df_prep.columns:
        df_prep = df_prep[df_prep[col].fillna(0) >= 0].copy()

numeric_cols = df_prep.select_dtypes(include=[np.number]).columns.tolist()
for col in numeric_cols:
    df_prep[col] = df_prep[col].fillna(df_prep[col].median())

object_cols = df_prep.select_dtypes(include=["object"]).columns.tolist()
for col in object_cols:
    mode_val = df_prep[col].mode(dropna=True)
    fill_val = mode_val.iloc[0] if len(mode_val) > 0 else "Unknown"
    df_prep[col] = df_prep[col].fillna(fill_val)

label_encoders = {}
categorical_cols = [c for c in df_prep.select_dtypes(include=["object"]).columns if
for col in categorical_cols:
    encoder = LabelEncoder()
    df_prep[col] = encoder.fit_transform(df_prep[col].astype(str))
    label_encoders[col] = encoder

continuous_features = [c for c in continuous_candidates if c in df_prep.columns]

print("Shape before preprocessing:", shape_before)
print("Shape after leakage handling:", shape_after_leakage)
print("Shape after full preprocessing:", df_prep.shape)
print("Columns after preprocessing:", df_prep.columns.tolist())
```

```
Shape before preprocessing: (100000, 28)
Shape after leakage handling: (12500, 28)
Shape after full preprocessing: (12500, 22)
Columns after preprocessing: ['Age', 'Occupation', 'Annual_Income', 'Monthly_Inhand_
Salary', 'Num_Bank_Accounts', 'Num_Credit_Card', 'Interest_Rate', 'Num_of_Loan', 'De
lay_from_due_date', 'Num_of_Delayed_Payment', 'Changed_Credit_Limit', 'Num_Credit_In
quiries', 'Credit_Mix', 'Outstanding_Debt', 'Credit_Utilization_Ratio', 'Credit_Hist
ory_Age', 'Payment_of_Min_Amount', 'Total_EMI_per_month', 'Amount_invested_monthly',
'Payment_Behaviour', 'Monthly_Balance', 'Credit_Score']
```

4. Lưu CSV Đã Làm Sạch

Lưu bộ dữ liệu đã tiền xử lý thành tệp CSV trung gian theo đúng yêu cầu bắt buộc của đề bài.

In [4]:
```
CLEAN_PATH.parent.mkdir(parents=True, exist_ok=True)
df_prep.to_csv(CLEAN_PATH, index=False)
print(f"Saved cleaned data to: {CLEAN_PATH}")
print("Saved shape:", df_prep.shape)
```

```
Saved cleaned data to: d:\data-mining-credit-score\data\processed\cleaned_credit_dat
a.csv
Saved shape: (12500, 22)
```

5. Nạp Lại CSV Đã Làm Sạch

Nạp lại tệp CSV đã làm sạch nhằm đảm bảo tính độc lập giữa giai đoạn tiền xử lý và giai đoạn mô hình hóa.

In [5]:
```
if not CLEAN_PATH.exists():
    raise FileNotFoundError(f"Cleaned dataset not found: {CLEAN_PATH}")

cleaned_df = pd.read_csv(CLEAN_PATH)
print("Reloaded cleaned shape:", cleaned_df.shape)
cleaned_df.head()
```

```
Reloaded cleaned shape: (12500, 22)
```

Out[5]:

	Age	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Cr
0	23.0	12	19114.12	1824.843333	3.0	
1	28.0	13	34847.84	3037.986667	2.0	
2	34.0	4	143162.64	12187.220000	1.0	
3	54.0	5	30689.89	2612.490833	2.0	
4	21.0	2	35547.71	2853.309167	7.0	

5 rows × 22 columns

6. Phân Tích Khám Phá Dữ Liệu (EDA)

Trình bày đầy đủ các thống kê mô tả bắt buộc: kích thước dữ liệu, kiểu dữ liệu, phân phối nhãn mục tiêu và thống kê min/max/mean của biến liên tục.

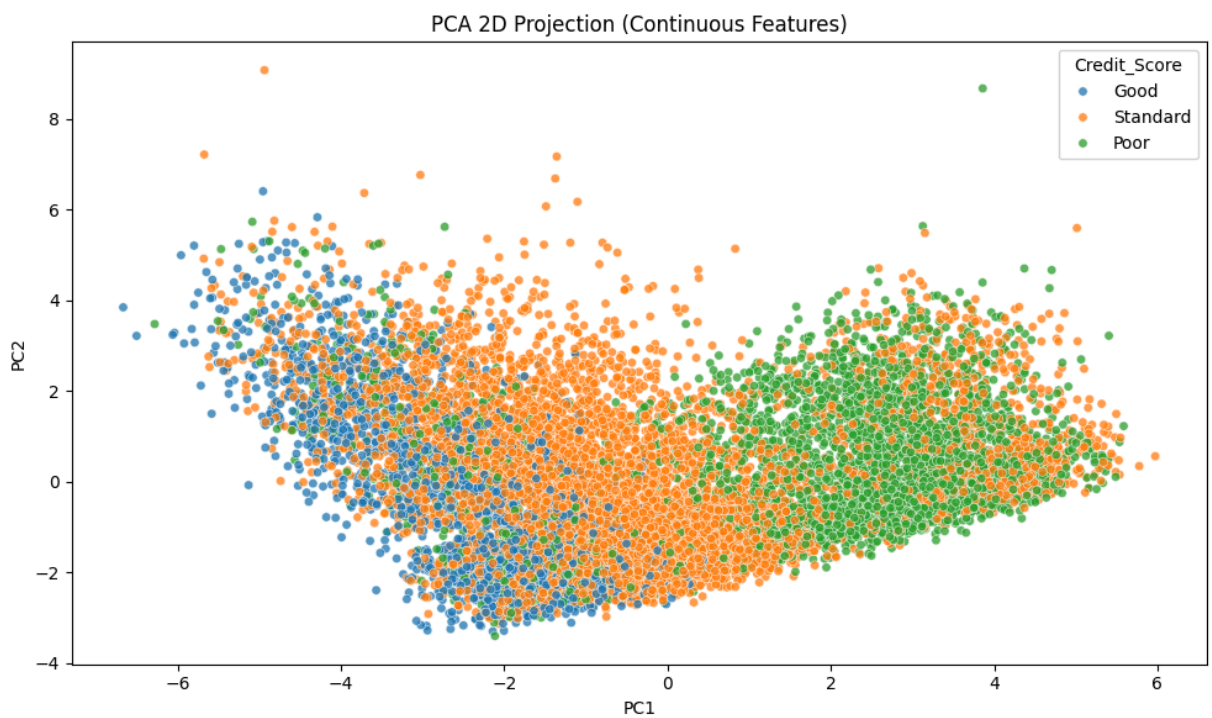
In [6]:
```
print("EDA - shape:", cleaned_df.shape)

print("\nEDA - dtypes (first 30):")
print(cleaned_df.dtypes.head(30))

if TARGET_COL in cleaned_df.columns:
    print("\nEDA - label distribution:")
    print(cleaned_df[TARGET_COL].value_counts(dropna=False))

eda_continuous_cols = [c for c in continuous_features if c in cleaned_df.columns]
if not eda_continuous_cols:
    eda_continuous_cols = [c for c in cleaned_df.select_dtypes(include=[np.number])

if len(eda_continuous_cols) > 0:
    desc_stats = cleaned_df[eda_continuous_cols].describe().T
    desc_stats = desc_stats[["min", "max", "mean"]].sort_index()
    print("\nEDA - descriptive stats (continuous features):")
    print(desc_stats)
else:
    print("No continuous columns available for descriptive statistics.")
```

```
EDA - shape: (12500, 22)

EDA - dtypes (first 30):
Age                         float64
Occupation                    int64
Annual_Income               float64
Monthly_Inhand_Salary       float64
Num_Bank_Accounts           float64
Num_Credit_Card             float64
Interest_Rate               float64
Num_of_Loan                 float64
Delay_from_due_date         float64
Num_of_Delayed_Payment      float64
Changed_Credit_Limit        float64
Num_Credit_Inquiries        float64
Credit_Mix                    int64
Outstanding_Debt            float64
Credit_Utilization_Ratio    float64
Credit_History_Age          float64
Payment_of_Min_Amount         int64
Total_EMI_per_month         float64
Amount_invested_monthly     float64
Payment_Behaviour             int64
Monthly_Balance             float64
Credit_Score                    str
dtype: object

EDA - label distribution:
Credit_Score
Standard    6943
Poor        3582
Good        1975
Name: count, dtype: int64

EDA - descriptive stats (continuous features):
                                  min            max          mean
Age                         14.000000      56.000000     33.037760
Amount_invested_monthly      0.000000     434.191089     55.101315
Annual_Income             7005.930000  179987.280000  50505.123449
Changed_Credit_Limit         0.500000      29.980000     10.482423
Credit_Utilization_Ratio    20.992914      49.564519     32.291949
Delay_from_due_date          0.000000      62.000000     21.064080
Interest_Rate                1.000000      34.000000     14.532080
Monthly_Balance              0.088628    1183.930696    395.831118
Monthly_Inhand_Salary      303.645417   15204.633333   4206.144676
Num_Bank_Accounts            0.000000      10.000000      5.367520
Num_Credit_Card              1.000000      10.000000      5.532400
Num_Credit_Inquiries         0.000000      17.000000      4.970720
Num_of_Delayed_Payment       0.000000      25.000000     13.306080
Num_of_Loan                  0.000000       9.000000      3.532880
Outstanding_Debt             0.230000    4998.070000   1426.220376
Total_EMI_per_month          0.000000    1705.000000     97.483206
```

7. Trực Quan Hóa PCA

Thực hiện PCA trên tập biến liên tục sau chuẩn hóa, giảm chiều về không gian 2D và trực quan hóa theo nhãn thực tế.

In [7]:
```
pca_features = [c for c in eda_continuous_cols if c != TARGET_COL]

if len(pca_features) < 2:
    print("Not enough continuous features for PCA(2).")
else:
    pca_scaler = StandardScaler()
    X_pca_scaled = pca_scaler.fit_transform(cleaned_df[pca_features])

    pca = PCA(n_components=2, random_state=SEED)
    X_pca_2d = pca.fit_transform(X_pca_scaled)

    pca_df = pd.DataFrame(X_pca_2d, columns=["PC1", "PC2"])
    if TARGET_COL in cleaned_df.columns:
        pca_df[TARGET_COL] = cleaned_df[TARGET_COL].astype(str).values

    plt.figure(figsize=(10, 6))
    if TARGET_COL in pca_df.columns:
        sns.scatterplot(data=pca_df, x="PC1", y="PC2", hue=TARGET_COL, alpha=0.75,
    else:
        sns.scatterplot(data=pca_df, x="PC1", y="PC2", alpha=0.75, s=28)
    plt.title("PCA 2D Projection (Continuous Features)")
    plt.tight_layout()
    plt.show()

    explained = pca.explained_variance_ratio_
    print(f"Explained variance ratio: PC1={explained[0]:.4f}, PC2={explained[1]:.4f
```




```
Explained variance ratio: PC1=0.3788, PC2=0.1767
```

8. Phân Lớp (3 Mô Hình Tune + 10-Fold CV)

Sử dụng ba mô hình phân lớp hợp lệ theo đề bài: KNN, Random Forest và SVM.

Không gian siêu tham số được tinh gọn nhằm giảm thời gian huấn luyện trên máy cá nhân, đồng thời vẫn bảo toàn GridSearchCV và đánh giá chéo 10-fold theo yêu cầu.

In [8]:
```
if TARGET_COL not in cleaned_df.columns:
    raise ValueError(f"Target column not found: {TARGET_COL}")

X_cls = cleaned_df.drop(columns=[TARGET_COL])
y_cls = cleaned_df[TARGET_COL].astype(str)

num_features = X_cls.select_dtypes(include=[np.number]).columns.tolist()
cat_features = [c for c in X_cls.columns if c not in num_features]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_features),
        ("cat", categorical_transformer, cat_features)
    ]
)

models = {
    "KNN": Pipeline(steps=[("preprocessor", preprocessor), ("classifier", KNeighbor
    "RandomForest": Pipeline(steps=[("preprocessor", preprocessor), ("classifier",
    "SVM": Pipeline(steps=[("preprocessor", preprocessor), ("classifier", SVC(rando
}

param_grids = {
    "KNN": {
        "classifier__n_neighbors": [5, 9],
        "classifier__weights": ["distance"],
        "classifier__p": [1, 2]
    },
    "RandomForest": {
        "classifier__n_estimators": [150, 250],
        "classifier__max_depth": [12, None],
        "classifier__min_samples_split": [2, 6]
    },
    "SVM": {
        "classifier__C": [0.5, 2],
        "classifier__kernel": ["linear"]
    }
}
```

```
cv10 = StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED)

if RUN_FULL_CLASSIFICATION:
    cls_rows = []
    for model_name, pipeline in models.items():
        print(f"Running GridSearchCV for {model_name}...")
        gs = GridSearchCV(
            estimator=pipeline,
            param_grid=param_grids[model_name],
            scoring="f1_macro",
            cv=cv10,
            n_jobs=-1,
            verbose=1
        )
        gs.fit(X_cls, y_cls)
        cls_rows.append({
            "Model": model_name,
            "Best_CV_F1_Macro": gs.best_score_,
            "Best_Params": gs.best_params_
        })

    classification_results = pd.DataFrame(cls_rows).sort_values("Best_CV_F1_Macro",
else:
    classification_results = pd.DataFrame({
        "Model": ["KNN", "RandomForest", "SVM"],
        "Best_CV_F1_Macro": [np.nan, np.nan, np.nan],
        "Best_Params": ["TODO (set RUN_FULL_CLASSIFICATION=True)"] * 3
    })
    print("Full classification is skipped. Set RUN_FULL_CLASSIFICATION=True to run

print(classification_results)
```

```
Running GridSearchCV for KNN...
Fitting 10 folds for each of 4 candidates, totalling 40 fits
Running GridSearchCV for RandomForest...
Fitting 10 folds for each of 8 candidates, totalling 80 fits
Running GridSearchCV for SVM...
Fitting 10 folds for each of 2 candidates, totalling 20 fits
          Model  Best_CV_F1_Macro  \
1  RandomForest          0.770607
2           SVM          0.707393
0           KNN          0.702710

                                        Best_Params
1  {'classifier__max_depth': 12, 'classifier__min...
2  {'classifier__C': 2, 'classifier__kernel': 'li...
0  {'classifier__n_neighbors': 9, 'classifier__p'...
```

9. Phân Cụm (KMeans + DBSCAN + ARI/NMI)

Thiết lập lưới tham số DBSCAN ở mức gọn để lựa chọn cấu hình phù hợp với chi phí tính toán thấp hơn trên máy cá nhân.

In [9]:
```
X_cluster = cleaned_df.drop(columns=[TARGET_COL], errors="ignore").copy()
y_true = cleaned_df[TARGET_COL].astype(str) if TARGET_COL in cleaned_df.columns els

for col in X_cluster.columns:
    if X_cluster[col].dtype == "object":
        X_cluster[col] = X_cluster[col].astype("category").cat.codes

X_cluster = X_cluster.replace([np.inf, -np.inf], np.nan)
X_cluster = X_cluster.fillna(X_cluster.median(numeric_only=True))

if RUN_FULL_CLUSTERING:
    cluster_scaler = StandardScaler()
    X_cluster_scaled = cluster_scaler.fit_transform(X_cluster)

    kmeans = KMeans(n_clusters=3, random_state=SEED, n_init=10)
    kmeans_labels = kmeans.fit_predict(X_cluster_scaled)

    dbscan_grid = [
        {"eps": 0.7, "min_samples": 8},
        {"eps": 0.9, "min_samples": 8},
        {"eps": 0.9, "min_samples": 12},
        {"eps": 1.1, "min_samples": 12}
    ]
    dbscan_records = []
    for cfg in dbscan_grid:
        labels = DBSCAN(eps=cfg["eps"], min_samples=cfg["min_samples"]).fit_predict
        if y_true is not None:
            ari_val = adjusted_rand_score(y_true, labels)
            nmi_val = normalized_mutual_info_score(y_true, labels)
        else:
            ari_val = np.nan
            nmi_val = np.nan
        dbscan_records.append({
            "eps": cfg["eps"],
            "min_samples": cfg["min_samples"],
            "labels": labels,
            "ARI": ari_val,
            "NMI": nmi_val
        })

    if y_true is not None:
        best_dbscan = max(dbscan_records, key=lambda x: x["ARI"])
    else:
        best_dbscan = dbscan_records[0]
    dbscan_labels = best_dbscan["labels"]

    dbscan_note = f"eps={best_dbscan['eps']}, min_samples={best_dbscan['min_samples

    if y_true is not None:
        clustering_results = pd.DataFrame([
            {
                "Algorithm": "KMeans",
                "ARI": adjusted_rand_score(y_true, kmeans_labels),
                "NMI": normalized_mutual_info_score(y_true, kmeans_labels),
                "Notes": "n_clusters=3"
```

```
            },
            {
                "Algorithm": "DBSCAN",
                "ARI": adjusted_rand_score(y_true, dbscan_labels),
                "NMI": normalized_mutual_info_score(y_true, dbscan_labels),
                "Notes": dbscan_note
            }
        ])
    else:
        clustering_results = pd.DataFrame([
            {
                "Algorithm": "KMeans",
                "ARI": np.nan,
                "NMI": np.nan,
                "Notes": "Target unavailable"
            },
            {
                "Algorithm": "DBSCAN",
                "ARI": np.nan,
                "NMI": np.nan,
                "Notes": dbscan_note + " (target unavailable)"
            }
        ])
else:
    clustering_results = pd.DataFrame([
        {
            "Algorithm": "KMeans",
            "ARI": np.nan,
            "NMI": np.nan,
            "Notes": "TODO (set RUN_FULL_CLUSTERING=True, n_clusters=3)"
        },
        {
            "Algorithm": "DBSCAN",
            "ARI": np.nan,
            "NMI": np.nan,
            "Notes": "TODO (set RUN_FULL_CLUSTERING=True, tune eps and min_samples)
        }
    ])
    print("Full clustering is skipped. Set RUN_FULL_CLUSTERING=True to run KMeans +

print(clustering_results)
```

```
  Algorithm       ARI       NMI                   Notes
0    KMeans  0.167158  0.191074            n_clusters=3
1    DBSCAN  0.000000  0.000000  eps=0.7, min_samples=8
```

10. Kết Luận

Tổng hợp các kết quả trọng tâm sau khi thực thi toàn bộ notebook:

- Chiến lược chống rò rỉ dữ liệu đã sử dụng và cơ sở lựa chọn.
- Nhận xét EDA quan trọng (mức độ mất cân bằng lớp, biên giá trị, chất lượng dữ liệu).
- Diễn giải kết quả PCA (mức độ tách biệt/chồng lấn giữa các lớp).
- Mô hình phân lớp tốt nhất theo F1-score và bộ siêu tham số tối ưu.

- So sánh hiệu quả KMeans và DBSCAN theo ARI/NMI.
- Kết luận tổng quát và các giới hạn của nghiên cứu.